BUSINESS VALUE · COST · ROI

Business case

What Kept is worth to Maple Bank, measured on the bank's own data

Team Sharpe Shooters · CIBC Collections Hackathon · 12 October 2026

$1.55M
a year, base case ($0.44M–4.3M)

2.4×
year-one ROI, base case

~2.6 mo
to pay back the build

+2.7 pts
promised money paid, measured on randomised cases

The short version. Collections loses money in one place: cases that *roll* into a worse bucket. The cheapest way to stop a roll is a promise the customer can actually keep. Measured on the bank's own randomised cases, within days-past-due bands, dating every promise to just after payday would raise promised money paid by **2.7 points** (95% CI 0.1 to 6.1; `docs/reports/evidence.md` §2). Kept dates promises to payday where it knows or can ask for it, contacts the people a contact actually helps, swaps letters for near-free digital nudges, and gives agents and QA reviewers time back. Counted conservatively, that's **$1.55M a year** against $150K to run and $300K to build, so it pays back in under three months. Every number below comes from the release, except seven assumptions that are named, explained and stress-tested in §7.

1 Where the money leaks

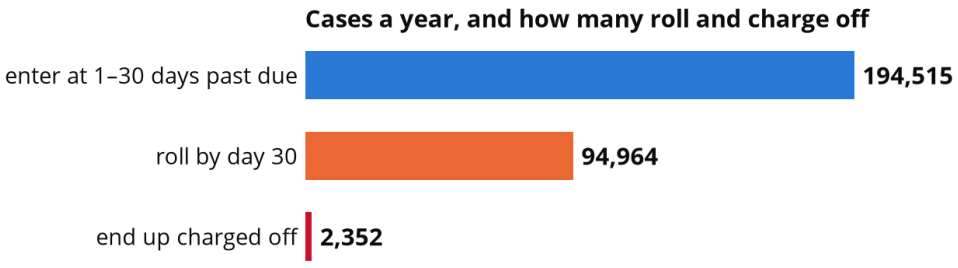


Figure 1. Cases that entered collections at 1–30 days past due in the last 12 months, and what happened next.

What the chart says. Of the 194,515 cases that entered at 1–30 days past due in the 12 months to 28 Sep 2026 ($63.9M overdue), about half roll into a worse bucket within 30 days (48.8% across 330,779 cases since 2016). Of the rolled cases, **2.5% end up charged off, at $12,523 on average**; of the cases that *don't* roll, **none** do. Rolling is the only road to a charge-off, so everything Kept does is aimed at the first 30 days.

2 The lever: timing the promise

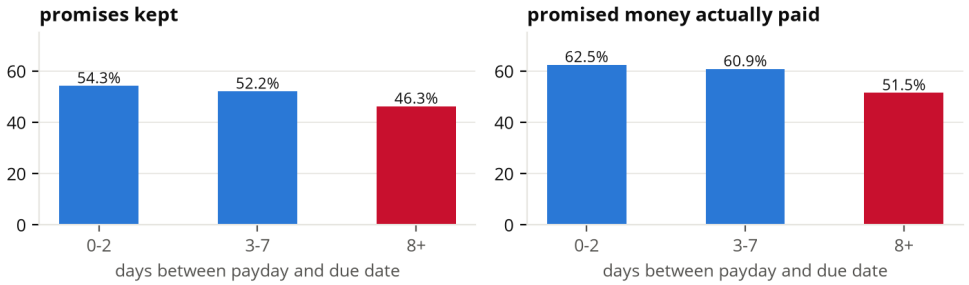


Figure 2. Resolved promises by days between the customer's last salary deposit and the due date.

What the chart says. Both panels slope the same way: the further a due date sits after payday, the worse the promise does. Due 0–2 days after payday, **54.3%** of promises are kept and **62.5%** of the promised money arrives. Due 8+ days after, when the pay is already spent, that drops to 46.3% and **51.5%**. The money panel moves more than the count panel, because a late-dated promise that does get paid is more often paid only in part.

Due date vs payday	Promises	Kept	Money paid	Share of promised $
0–2 days after	8,359	54.3%	62.5%	26%
3–7 days after	7,553	52.2%	60.9%	24%
8+ days after	16,045	46.3%	51.5%	50%

Table 1. Where we can see the customer's salary (32,000 resolved promises).

From the chart to dollars. The chart is descriptive: part of the gap is *who* gets offered which date. So the money uses the effect **measured on the bank's randomised cases**, within days-past-due bands: dating every promise to just after payday raises the promised money paid by **2.7 points** (95% CI 0.1 to 6.1). On the **$41.7M** promised in the last 12 months, for the 55% of promises whose payday Kept knows or is told, that's **$0.62M** a year. The 5% hold-out measures the real effect after launch.

How Kept uses this

Kept sizing is plain arithmetic, not a model: the due date is 1–2 days after the expected payroll date (detected from salary deposits), and the amount is split into up to three instalments that each fit inside the customer's spare cash for that pay period. A promise that can't be sized that way isn't proposed at all.

3 A cheaper, kinder contact mix

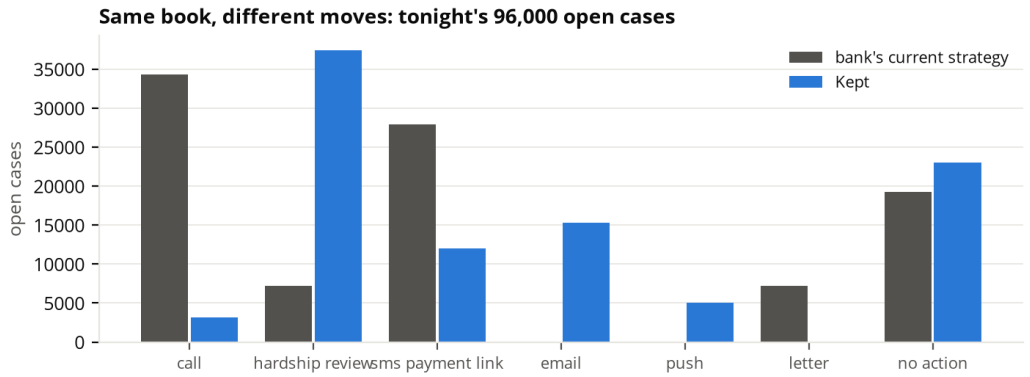


Figure 3. Tonight's 96,000 open cases: the bank's current strategy vs Kept.

What the chart says. Kept doesn't simply call less: call-type actions are almost unchanged (40,601 vs 41,574). What changes is *which* calls. The current strategy's biggest bar is collection calls (34,326); Kept's is **hardship reviews (37,409)**, because text and cash-flow signals flag far more customers who need a protective conversation than the old rule set did. At the cheap end, $1.10 letters (7,180) disappear in favour of SMS payment links ($0.01), emails and app pushes ($0.001).

	Cost per contact	Current strategy	Kept
Call-type (call, promise, plan, hardship review)	$1.75	41,574	40,601
Letter	$1.10	7,180	0
SMS payment link	$0.01	27,942	11,971
Email · app push	$0.001	0	20,370
Cost of one pass over the open book		**$80,932**	**$71,192**

Table 2. Channel costs are the release's own averages for September 2026.

Each pass over the book saves **$9,740**. The open book is worked through roughly weekly under the 3-contacts-in-7-days cap, so 52 passes a year give **$506K**.

What the model expects. The uplift model estimates that tonight's 55,674 positive-uplift proposals bring in **$2.06M more within 14 days** than doing nothing. That's a model estimate, so it's *not* in our totals; it's exactly what the hold-out is there to check.

4 Time back for agents and QA

	Base	How it's computed
Call summaries	$378K	629,435 agent calls a year × 60 s of after-call work saved × $36/h loaded agent cost (≈ 5.8 agents' time back on the phones)
QA pre-scoring	$45K	today's 2% manual sample is 2,518 reviewer-hours a year; scoring every call first lets reviewers skip the clean ones (50% saved) and coverage goes from 2% of calls to all of them

Table 3. Average handle time in the release is 6.8 minutes, so 60 seconds of wrap-up is a modest assumption.

5 The upside we don't count: roll rate

From §1: 194,515 cases a year enter at 1–30 days past due, 2.5% of the ones that roll get charged off, at $12.5K each. So **every point of 30-day roll rate is worth about $603K a year** in charge-offs avoided, before counting the collection cost of the rolled cases. Our design hypothesised 1–2 points.

We keep it **out of the base case**: a kept promise *is* what stops a roll, so counting both would count the payday effect twice. The high case adds one point.

6 Putting it together

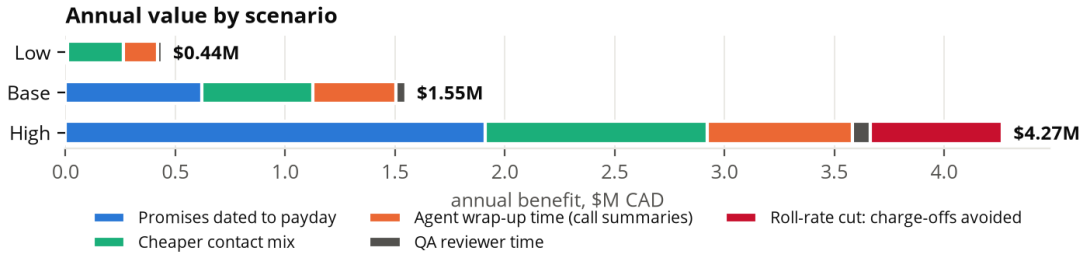


Figure 4. Annual value by component and scenario.

What the chart says. In every scenario the payday effect is the largest block, and the contact mix the second. The low case takes the payday effect at the bottom of its confidence interval with only the bank's own payday data, and halves the book passes and the time saved: $0.44M a year, which doesn't pay back the build within year one. The high case adds the roll-rate point and the top of the interval.

Why it's cheap to run. One nightly batch scores the whole book in about seven minutes on a single machine. Every language-model call, the Ask included, runs on a local open model on one GPU: no per-call fees, and customer text never leaves the bank. DuckDB and dbt replace a cluster. The $150K is mostly one operations engineer ($120K) plus about $30K of GPU server

Annual value (CAD)	Low	Base	High
Promises dated to payday	10,018	619,866	1,909,690
Cheaper contact mix	253,239	506,479	1,012,957
Agent wrap-up time (call summaries)	157,359	377,661	660,907
QA reviewer time	18,883	45,319	79,309
Roll-rate reduction (charge-offs avoided)	0	0	603,362
Total benefit a year	**439,499**	**1,549,325**	**4,266,225**
Run cost a year	150,000	150,000	150,000
Build, one-off	300,000	300,000	300,000
Year-one ROI	**-0.0×**	**2.4×**	**8.5×**
Payback	12.4 months	2.6 months	0.9 months

Table 4. Year-one ROI = (benefit − run − build) ÷ (run + build). Payback = build ÷ (benefit − run), in months.

and warehouse compute. The $300K build is four engineers for three months.

7 What moves the number

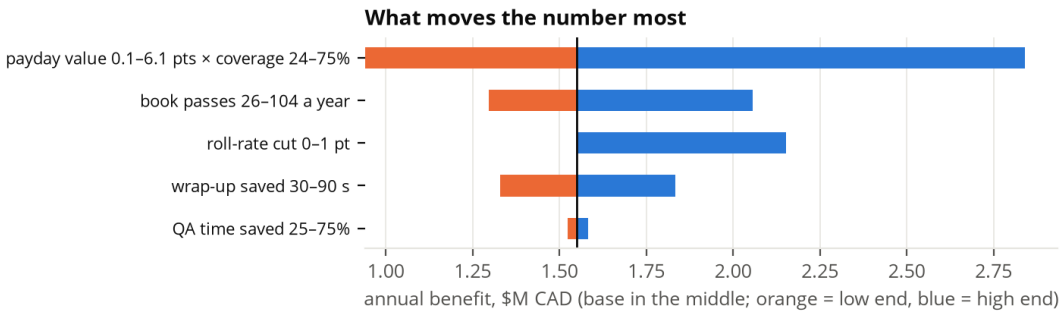


Figure 5. Each assumption swung from its low to its high value, everything else at base.

What the chart says. The answer hinges on one question: **how big is the payday effect, and for how many promises can Kept know the payday?** Together they swing the total by about $1.9M. The next biggest are how often the book is worked (contact savings) and the roll-rate point. Minutes saved and QA time barely move it. That's why the rollout plan is built around measuring the payday effect first.

8 Break-even, and the chance of paying back

Break-even. The year-one cost is $450K (build plus a year's running). At base, the contact mix, wrap-up time and QA time alone bring in $0.93M, so **Kept pays back in year one even if dating promises to payday adds nothing**. With every one of those at its low end as well, it needs a payday effect of **0.2 points**; the measured effect is 2.7 points [0.1, 6.1] (evidence §2).

The chance of paying back. Instead of three scenarios, draw every assumption at once, 20,000 times: each from a triangular distribution over its low, base and high values, and the payday

Assumption	Low	Base	High	Why this range
Payday effect (measured)	0.1 pts	2.7 pts	6.1 pts	95% interval, randomised cases (evidence §2)
Promises Kept can date	24%	55%	75%	bank's payday data; plus what customers state; plus the pay page asking
Passes over the book a year	26	52	104	weekly work-through under the contact cap
After-call work saved	30 s	60 s	90 s	the summary pre-fills the wrap-up note
Loaded agent cost	$30/h	$36/h	$42/h	~$50K salary + 30% on-cost
QA reviewer time saved	25%	50%	75%	reviewers skip pre-scored clean calls
Roll-rate points counted	0	0	1	overlaps the payday effect

Table 5. The assumptions in the model, with the measured payday effect's interval. Everything else is measured.

effect from a normal distribution on its measured interval (so a zero or negative effect is possible).

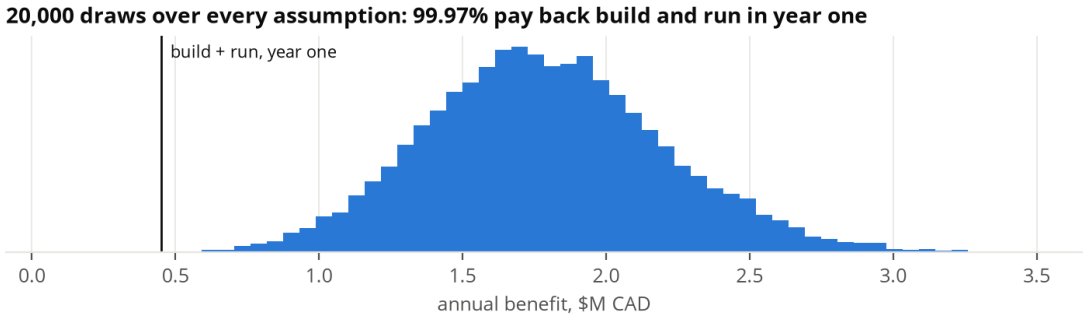


Figure 6. Annual benefit across 20,000 draws of every assumption. The line is the year-one cost, build plus run.

What the chart says. The median annual benefit is $1.77M, and 19 draws in 20 fall between $1.13M and $2.50M. **99.97% of draws pay back build and run within year one.** The draws treat the assumptions as independent, which narrows the spread. The strictest check runs the same draws with every assumption moving together (all high or all low at once): 19 in 20 then fall between $0.64M and $3.45M, and **98.5% still pay back within year one**. The low scenario ($0.44M) is the corner where every assumption is at its worst at the same time.

9 Risks, costs we're honest about, and how we'll know

- **Specialist workload.** 38,187 of the 96,000 cards (40%) were Red, and every one carried a hardship signal: 25,914 from the case's own hardship flag, 12,273 from what notes, calls or the customer said, and 9,865 of them also marked vulnerable. A specialist reviews each Red card before anything is sent. That is the protection working, but 7,367 of them were customers whose hardship plan was still running, where a new review repeats the last one. Kept now skips the review while a plan runs (the card says it is monitoring the plan, and every protection and hard stop still applies): Red cards fell to 33,646, about 750 specialist hours a pass saved at 10 minutes a review.
- **Not yet a trial.** The payday effect is measured on randomised cases and checked three ways

(evidence §2a), but the due dates themselves weren't randomised, so it is still an estimate. A permanent **hold-out** stays on the old strategy and a **kill switch** reverts everyone if the effect isn't there. How long the hold-out needs (80% power, 5% significance, 3,720 new 1–30 day cases and 2,720 promises a week):

To detect	5% hold-out	10%	20%
Promises kept +4.6 pts (the measured effect)	7 weeks	4 weeks	2 weeks
Promises kept +2 pts	35 weeks	19 weeks	11 weeks
Roll rate −2 pts	28 weeks	15 weeks	8 weeks
Roll rate −1 pt	111 weeks	59 weeks	33 weeks

So the plan is four weeks in shadow mode, then a 20% hold-out for eight weeks, which shows the payday effect within two weeks and a 2-point roll-rate drop by week eight, then 5% once it is proven.

- **What isn't in the numbers.** Fewer complaints and less regulator risk from finding hardship when the customer says it rather than weeks later; fairness checked on every run; a plain-English reason and an audit trail for every decision. Those are hard to price and matter anyway.

Numbers `scripts/business_case.py` reads only the release files and the scored cards, and caches the numbers in `data/exports/reports/business_data.json`.

Rebuild `uv run python scripts/reportlib.py business_case`